



CLUB CODE



POSTMAN

J W T

Express

JS

API Backend Development

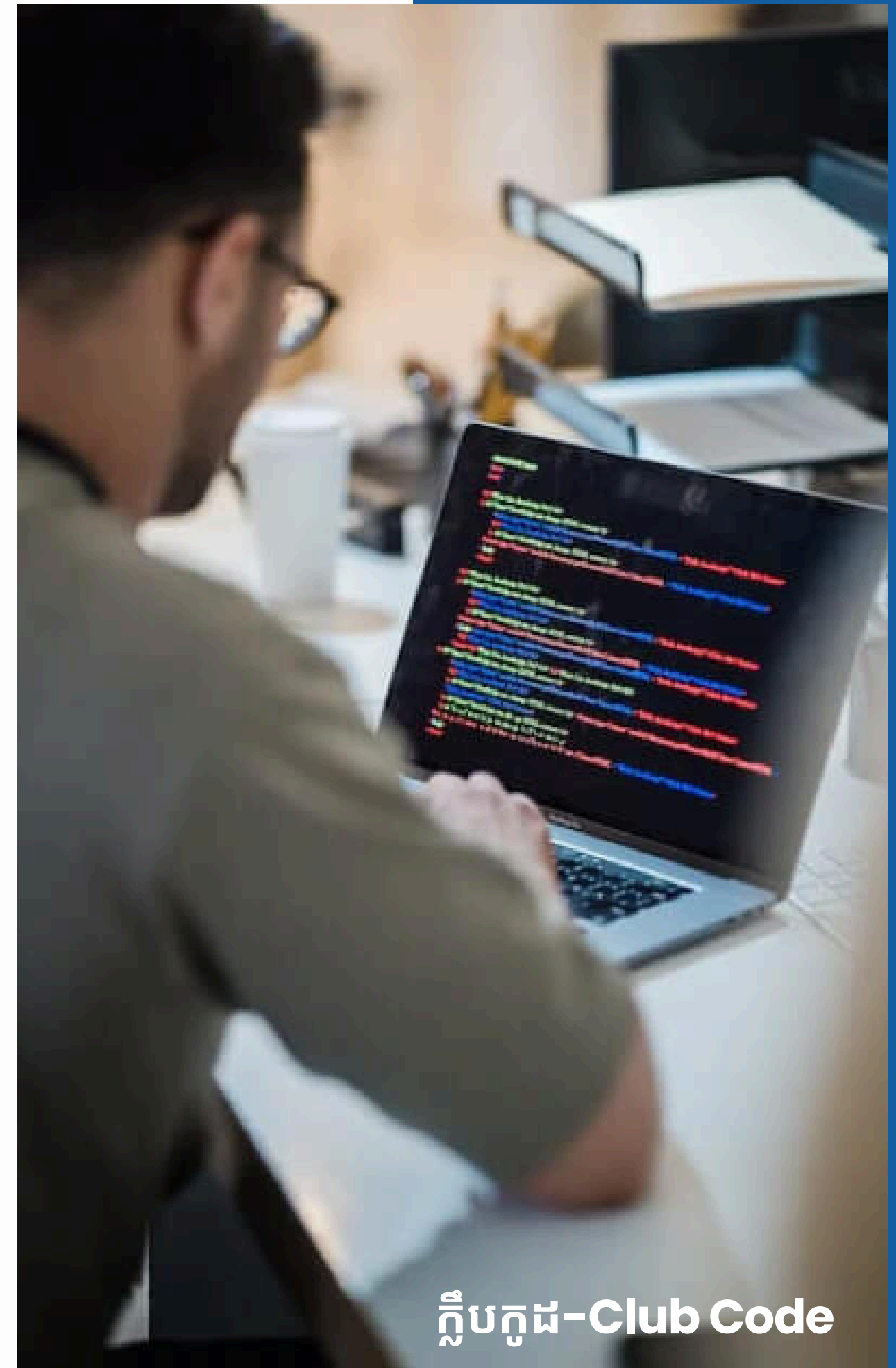


Mr. PHORN KHOUCH
WEB APPLICATION DEVELOPER



Contents

- ✓ [Introduction & installation](#)
- ✓ [Node.js Basic console](#)
- ✓ [Import & export module](#)
- ✓ [Plugin express.js](#)
- ✓ [Route and controller](#)
- ✓ [nodemon](#)
- ✓ [http method + Postman](#)
- ✓ [Connect MYSQL DB](#)
- ✓ [File Error Log](#)
- ✓ [bcrypt](#)
- ✓ [Plugin JWT & refresh Token](#)
- ✓ [JWT authorization](#)
- ✓ [Project Course](#)



Introduction

Node.js គឺជា runtime environment JavaScript នៅផ្នែកខាងម៉ាស៊ីនបម្រើ (server-side) ។ វាត្រូវបានបង្កើតឡើងលើ V8 JavaScript engine (ដែលប្រើនៅក្នុង Google Chrome) ហើយអាចអនុញ្ញាតឱ្យអ្នកសរសេរកូដ JavaScript ដើម្បីបង្កើតកម្មវិធី backend ឬ API បាន។

💡 លក្ខណៈពិសេស (Features)

- ✓ សរសេរជា JavaScript ប៉ុណ្ណោះ
- ✓ Node.js លឿនព្រោះវាធ្វើការ non-blocking និង asynchronous (មិនរងចាំការងាររួច)។
- ✓ ល្អសម្រាប់បង្កើត Web App និង API,...

Installation Step: Node JS Express JS

01

Step 1: Download Node.js

ចូលទៅកាន់គេហទំព័រ៖ <https://nodejs.org>
ជ្រើសយក LTS version (យើងផ្តល់អាទិភាពសុវត្ថិភាព)

02

Step 2: Install Node.js

- បន្ទាប់ពីទាញយករួច ដំណើរការ .msi file
- ចុច "Next" ច្រើនដង ទុក Default settings
- នៅពេលដល់ "Install Tools for Native Modules" → អាច uncheck ក៏បាន (optional)
- ចុច "Install" → រង់ចាំឲ្យវាដំឡើងរួច

03

Step 3: Check Node.js is installed

បើក Command Prompt ឬ PowerShell, បញ្ចូល៖

- `node -v` : version node js
- `npm -v` : version npm package manager

04

Step 4: Install Express.js

បើក Command Prompt ឬ PowerShell, បញ្ចូល៖

- `npm install express`

Basic console

មុខងារ Console ដែលមានប្រយោជន៍

Function	អត្ថន័យ
<code>console.log()</code>	បង្ហាញអ្វីៗធម្មតា
<code>console.error()</code>	បង្ហាញកំហុស
<code>console.warn()</code>	បង្ហាញការព្រមាន
<code>console.table()</code>	បង្ហាញអាន ឬ Object ជាតារាង
<code>console.time()</code> / <code>console.timeEnd()</code>	វាស់ពេលវេលាដំណើរការ

Basic console Example

js

```
console.log("ការបង្ហាញធម្មតា");
console.warn("នេះជាព្រមាន");
console.error("មានបញ្ហា!");

const people = [
  { name: "Dara", age: 25 },
  { name: "Vuthy", age: 30 }
];
console.table(people);

console.time("loop");
for (let i = 0; i < 1000000; i++) {}
console.timeEnd("loop");
```

Output Example:

bash

ការបង្ហាញធម្មតា
នេះជាព្រមាន
មានបញ្ហា!

(index)	name	age
0	'Dara'	25
1	'Vuthy'	30

loop: 4.271ms

Module import & export

Module ក្នុង Node.js គឺជាអ្វី?

Module ជាប្រភេទឯកសារ JavaScript មួយ (ឬភាគនៃកូដ) ដែលអាចនាំចូល (import) និងនាំចេញ (export) ដើម្បីប្រើប្រាស់ឡើងវិញ។ វាជាជំហានសំខាន់សម្រាប់ការបែងចែកកូដអោយមានរបៀប និងងាយស្រួលថែទាំ។

📤 Export Module (នាំចេញ Module)

នៅក្នុងឯកសារ .js មួយ អ្នកអាចនាំចេញ function, variable, object ឬ class ដោយប្រើ module.exports ឬ exports។

ឧទាហរណ៍៖ greet.js

js

Copy

Edit

```
function sayHello(name) {  
  return `សួស្តី ${name}`;  
}  
  
// export function  
module.exports = sayHello;
```


Module import & export

📥 Import Module (នាំចូល Module)

អ្នកអាចនាំចូល module មកប្រើដោយ require() នៅក្នុងឯកសារ JavaScript ផ្សេងទៀត។

ឧទាហរណ៍៖ app.js

```
js                                     Copy Edit

const sayHello = require('./greet');

console.log(sayHello("Dara")); // Output: ស្អែក Dara
```

📦 លទ្ធផលនៃការប្រើ Module

- ជួយធ្វើអោយ project ងាយគ្រប់គ្រង
- អាច reuse កូដបានគ្រប់ទីកន្លែង
- ជាផ្នែកមួយនៃការសរសេរកូដដែលមានស្តង់ដារ (Modular Coding)

Plugin Express js

Express.js ជាអ្វី?

Express.js គឺជា framework/lightweight plugin សម្រាប់ Node.js ដែលជួយឱ្យអ្នកអាចបង្កើត web application ឬ REST API បានយ៉ាងឆាប់រហ័ស និងងាយស្រួល។

របៀបដំឡើង *Express.js*

bash

 Copy

 Edit

```
npm init -y          # បង្កើត project Node.js
npm install express   # ដំឡើង Express.js
```

មុខងារសំខាន់ៗនៃ Express.js

app.get()

: ចាប់សំណើ HTTP GET

app.post()

: ចាប់សំណើ HTTP POST

app.use()

: ប្រើ Middleware ឬ static files

req និង res

: req ជា object សម្រាប់សំណើ client, res ជាសម្រាប់ឆ្លើយតប

app.listen()

: បង្កើត server ដើម្បីស្តាប់ការភ្ជាប់នៅលើ port

ក្លឹបកូដ-Club Code

Middleware Example

js

Copy

Edit

```
app.use(express.json());

app.post('/user', (req, res) => {
  const name = req.body.name;
  res.send(`សូម្បី ${name}`);
});
```

Send data with [Postman](#) or [curl](#):

json

Copy

Edit

```
{
  "name": "Rotha"
}
```

✓ លទ្ធផលនៃការប្រើ Express.js

- កូដសាមញ្ញ ច្បាស់លាស់
- វែងឆ្ងាយទៅ production-ready app ងាយ
- មាន ecosystem ធំ (middleware, plugin, router, security)

ក្លឹបកូដ-Club Code

Route & Controller

- **Route** គឺជា “ផ្លូវ” ឬ “URL path” ដែលបើកប្រើចូល (សម្រាប់ GET, POST, PUT, DELETE...) វានឹងបញ្ជូនទៅមុខងារក្នុង server។

```
js
app.get('/hello', (req, res) => {
  res.send('សូម្បីពី route /hello');
});
```

Copy

Edit

- ប្រើ GET request
- នៅពេលចូលទៅកាន់ <http://localhost:3000/hello> នឹងឆ្លើយតបជាសារ
- **Controller** គឺជាផ្នែកដែលដំណើរការក្នុង logic នៅពីក្រោយ Route — ដូចជា query database, validate data, បំប្លែង JSON, ធ្វើសារឆ្លើយតប...
 - “Route គឺជាផ្លូវ”
 - “Controller គឺជាអ្នកដឹកនាំលើផ្លូវនោះ”

Basic Example

បកប្រែ-Club Code

1. userController.js

```
js
// controllers/userController.js
exports.getUser = (req, res) => {
  res.send('ព័ត៌មានអ្នកប្រើ');
};

exports.createUser = (req, res) => {
  const name = req.body.name;
  res.send(`អ្នកប្រើ ${name} ត្រូវបានបង្កើត`);
};
```

Copy Edit

2. userRoutes.js

```
js
// routes/userRoutes.js
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');

// Define routes
router.get('/', userController.getUser);
router.post('/', userController.createUser);

module.exports = router;
```

Copy Edit

3. app.js

```
js
// app.js
const express = require('express');
const app = express();
const userRoutes = require('./routes/userRoutes');

app.use(express.json());

// Use routes
app.use('/users', userRoutes);

app.listen(3000, () => {
  console.log('Server កំពុងដំណើរការ');
});
```

Nodemon

- **Nodemon** ជា tool មួយសម្រាប់ Node.js ដែលអាច ចាប់សញ្ញាប្រែប្រួលក្នុងកូដ ហើយ រត់ server ឡើងវិញដោយស្វ័យប្រវត្តិ។ នេះមានប្រយោជន៍ណាស់ក្នុងពេលអភិវឌ្ឍ (development) ដោយមិនចាំបាច់ Ctrl + C ហើយ node app.js ទៅវិញទៅមកទេ។

📥 របៀបដំឡើង Nodemon

```
bash
npm install -g nodemon
```

Copy Edit

Note: -g មានន័យថា ដំឡើងជា global (ប្រើបានគ្រប់ project)

▶ របៀបប្រើប្រាស់ Nodemon

```
bash
nodemon app.js
```

Copy Edit

ពេលអ្នកធ្វើការកែប្រែក្នុង app.js, nodemon នឹង restart ដោយស្វ័យប្រវត្តិ។

HTTP Methods

HTTP Methods គឺជាវិធីសាស្ត្រសំរាប់ធ្វើសកម្មភាពផ្សេងៗ (CRUD) នៅលើទិន្នន័យក្នុង API។
យើងមានមធ្យោបាយចម្បងដូចខាងក្រោម៖

Method	អត្ថន័យ	ឧទាហរណ៍
GET	អានទិន្នន័យ	ទៅយក users, products, ...
POST	បញ្ចូលទិន្នន័យថ្មី	បង្កើតអ្នកប្រើថ្មី
PUT	កែប្រែទិន្នន័យ	កែអំពីអ្នកប្រើមួយ
DELETE	លុបទិន្នន័យ	លុបអ្នកប្រើ

Connect MYSQL DB

អ្វីដែលត្រូវមាន

- ✓ Node.js + Express.js
- ✓ MySQL Server ត្រូវដំឡើង
- ✓ Table ឧទាហរណ៍ (ex: users)
- ✓ VS Code/Postman/Simple DB tool (phpMyAdmin, MySQL Workbench...)

ដំឡើង MySQL Module

```
bash

npm install mysql2
```

Copy Edit

mysql2 គឺជា library modern និង support async/await

ការភ្ជាប់ MySQL

```
js

const mysql = require('mysql2');

// Create the connection
const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: '', // ដាក់ពាក្យសម្ងាត់ DB របស់អ្នក
  database: 'testdb', // ឈ្មោះ DB របស់អ្នក
});

module.exports = pool.promise(); // Use promises for async/await
```

Copy Edit

File Error Log

📄 គោលបំណង

- ✓ បង្រៀនរបៀបសរសេរ error log ទៅក្នុង .log file
- ✓ ប្រើ module fs (filesystem)
- ✓ មាន datetime និង message error

📁 **errorLogger.js** – function សរសេរ error ទៅ log file

```
js
const fs = require('fs');
const path = require('path');

const logPath = path.join(__dirname, 'logs', 'error.log');

function logError(error) {
  const time = new Date().toISOString();
  const message = `[${time}] ERROR: ${error}\n`;

  fs.appendFile(logPath, message, (err) => {
    if (err) {
      console.error('⚠ Failed to write log:', err.message);
    }
  });
}

module.exports = logError;
```

Basic Example

📁 app.js – API + log error to file

```
js
const express = require('express');
const app = express();
const logError = require('./errorLogger');

app.get('/', (req, res) => {
  res.send('ស្វាគមន៍ពី Node.js + Logging!');
});

// Example Error API
app.get('/error', (req, res) => {
  try {
    throw new Error("មានបញ្ហាក្នុងការទាញទិន្នន័យ!");
  } catch (err) {
    logError(err.message);
    res.status(500).send('មានបញ្ហាខាងក្នុង Server!');
  }
});

app.listen(3000, () => {
  console.log('🚀 Server ដំណើរការ http://localhost:3000');
});
```

🔧 **Result:**
ក្នុង folder logs/error.log អ្នកនឹងឃើញអ្វីដូចជា៖

```
vbnet
[2025-04-13T08:45:30.123Z] ERROR: មានបញ្ហាក្នុងការទាញទិន្នន័យ!
[2025-04-13T08:47:12.456Z] ERROR: Failed to connect to database
```

✅ អត្ថប្រយោជន៍

- អាច Debug កំហុសក្រោយបាន
- មិនបង្ហាញ message ពេញទៅ client
- អាចអនុវត្តបានទាំង production & local

bcrypt

- bcrypt គឺជា library សម្រាប់ Hash ពាក្យសម្ងាត់ (encrypt password)
- បម្រុងការពារ password
- អាច Verify password នៅពេល Login

📦 ដំឡើង bcrypt

```
bash
```

📋 Copy

✎ Edit

```
npm install bcrypt
```

✅ អត្ថប្រយោជន៍

លក្ខណៈ:	អត្ថន័យ
🔒 Hash	លាក់ពាក្យសម្ងាត់ពិតក្នុង DB
🕒 Salt Rounds	បន្ថែមកម្លាំង Hash (កុំឲ្យ Hacker brute force)
⚖️ Secure	ការពារព័ត៌មានសម្ងាត់របស់អ្នកប្រើ

Sample use

🔗 ការប្រើ bcrypt នៅក្នុង Node.js

✓ Hash Password (Register)

```
js
const bcrypt = require('bcrypt');

async function registerUser(password) {
  const saltRounds = 10;
  const hashedPassword = await bcrypt.hash(password, saltRounds);
  console.log("🔒 Password hashed:", hashedPassword);

  // Save hashedPassword to DB instead of plain password
}

registerUser("mypassword123");
```

Copy

Edit

Sample use

👉 ការប្រើ bcrypt នៅក្នុង Node.js

✅ Compare Password (Login)

```
js
async function loginUser(inputPassword, storedHashedPassword) {
  const isMatch = await bcrypt.compare(inputPassword, storedHashedPassword);

  if (isMatch) {
    console.log("✅ Password ត្រឹមត្រូវ!");
  } else {
    console.log("❌ Password មិនត្រឹមត្រូវ!");
  }
}

// Example usage
const hashed = '$2b$10$xJz8vEjB1k6W0L8xG9Mep.vdrkB4sBP.ha2ZIQDsAf0aT4pj3fV5W'; // hash of 'mypassword'
loginUser("mypassword123", hashed);
```


JWT (JSON Web Token)

JWT គឺជា Token ដើម្បីបញ្ជាក់ថា “អ្នកប្រើបាន Authenticate”

- នៅពេល login ជោគជ័យ អ្នកប្រើនឹងទទួលបាន:
- Access Token – តិចពេល (ex: 15mn), សម្រាប់ API
- Refresh Token – វែងពេល (ex: 7 days), សម្រាប់ renew access token

🔧 ដំឡើង Plugin ចាំបាច់

```
bash
```

Copy

Edit

```
npm install jsonwebtoken dotenv
```

jsonwebtoken – បង្កើត និង Verify JWT

dotenv – កំណត់ secret នៅក្នុង .env file

JWT authorization

- JWT Authorization ជាដំណើរការប្រើ JSON Web Token (JWT) ដើម្បី

ពិនិត្យសិទ្ធិអ្នកប្រើ បន្ទាប់ពីអ្នកប្រើបាន Login (Authenticated) រួច ដើម្បីកំណត់អត្តសញ្ញាណ អ្នកចូលប្រើប្រាស់។

របៀប Apply JWT Authorization

- បង្កើត JWT Token (មាន role ឬ permission)
- Middleware បញ្ជាក់ JWT + បញ្ជាក់តួនាទី
- ឧទាហរណ៍: admin អាច delete post, user មិនអាច

Middleware: Check Role (Authorization)

js

Copy

Edit

```
function authorizeRole(allowedRoles = []) {
  return (req, res, next) => {
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ message: "🚫 អ្នកមិនមានសិទ្ធិ!" });
    }
    next();
  };
}
```

THANK YOU!



PHORN KHOUCH

Web Application Developer

 096 80 77 189

 www.club-code.netlify.app

 Phnom Penh , Cambodai

